

Département IEM (*Informatique Electronique Mécanique*)

M1 ESI : *Electronique, Signal et Image - Semestre 2*

Programmation en langage Python

Travaux pratiques (10h)



Document préparé par :

Mahdi Madani & Abdellah El-Zaar

Contact : mahdi.madani@u-bourgogne.fr

Cours en ligne : <https://plubel-prod.u-bourgogne.fr/course/view.php?id=6335>

Sommaire

1. Guide d'utilisation de l'environnement de travail Anaconda Navigator
 - 1.1. Ouvrir et activer l'environnement
 - 1.2. Lancer JupyterLab
 - 1.3. Créer un notebook
 - 1.4. Coder, exécuter et sauvegarder son travail
2. TP1 : La récursivité, le tri et la recherche dichotomique en Python
 - 2.1. La récursivité
 - 2.2. La recherche dichotomique
 - 2.3. Les algorithmes de tri
3. TP2 : Le chiffrement de César
 - 3.1. Partie I : chiffrement d'un texte clair
 - 3.2. Partie II : reconstitution du texte clair à partir d'un texte chiffré
 - 3.3. Créer une application graphique sous Tkinter
4. TP3 : La programmation orientée objets
 - 4.1. Organiser et encapsuler son code en classes et méthodes
 - 4.2. Instancier et héritage d'une classe
 - 4.3. Héritage multiple
5. TP4 : Le traitement d'une image numérique
 - 5.1. Introduction du TP
 - 5.2. Travail à réaliser
6. TP5 : L'analyse de données à partir d'un fichier tableur
 - 6.1. Introduction du TP
 - 6.2. Travail à réaliser

Guide d'utilisation de l'environnement de travail Anaconda Navigator

Vos TP's vont se faire sur la plateforme Anaconda Navigator qui vous donne accès à la programmation de scripts en langage R ou Python mais aussi d'accéder à une large collection de paquets open-source et disponible sous linux, macOS ou Windows. Il a été choisi pour vos travaux grâce à son interface visuelle permettant de lancer des API et de gérer les environnements et elle propose un espace de développement flexible et efficace, tout en réduisant les défis liés à la configuration et à la gestion des dépendances.

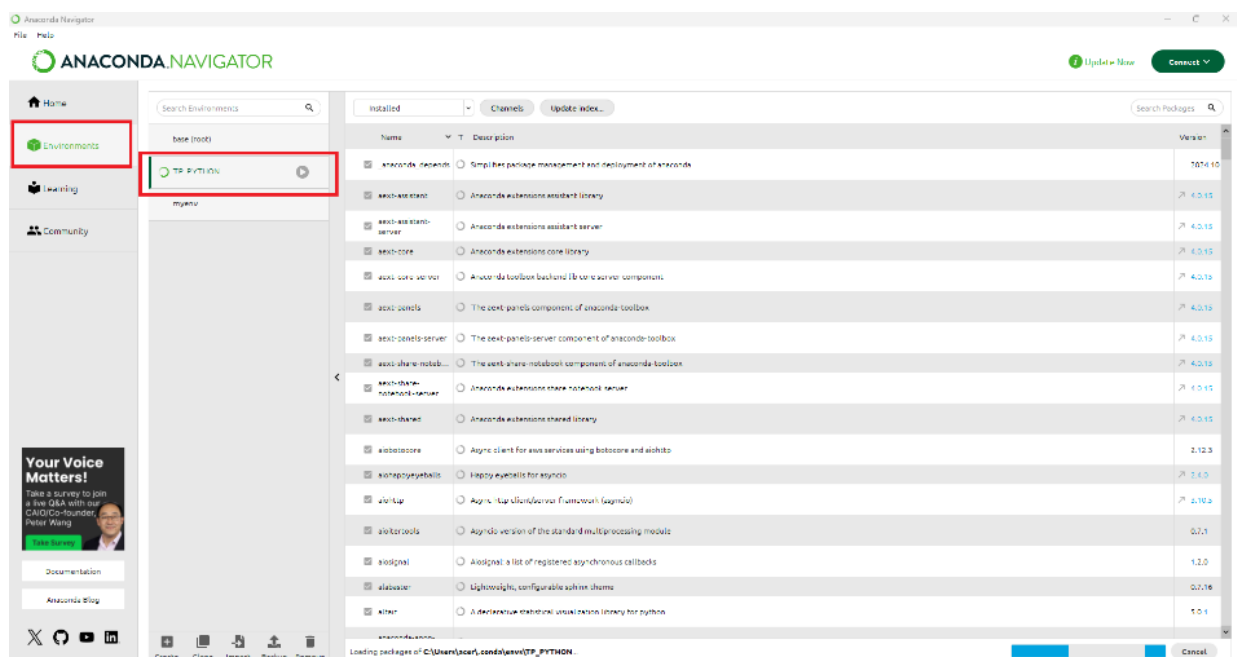
On commence donc ce fascicule par un petit tutorial vous guidant à créer votre première application et la lancer sous JupyterLab qui est l'interface qui vous permet d'écrire et d'exécuter vos scripts python dans des notebooks qui ont l'extension '*.ipynb'.

1. Ouvrir Anaconda Navigator

1. Cliquez sur **Anaconda Navigator** depuis votre bureau.
2. Patientez pendant que l'application se charge.

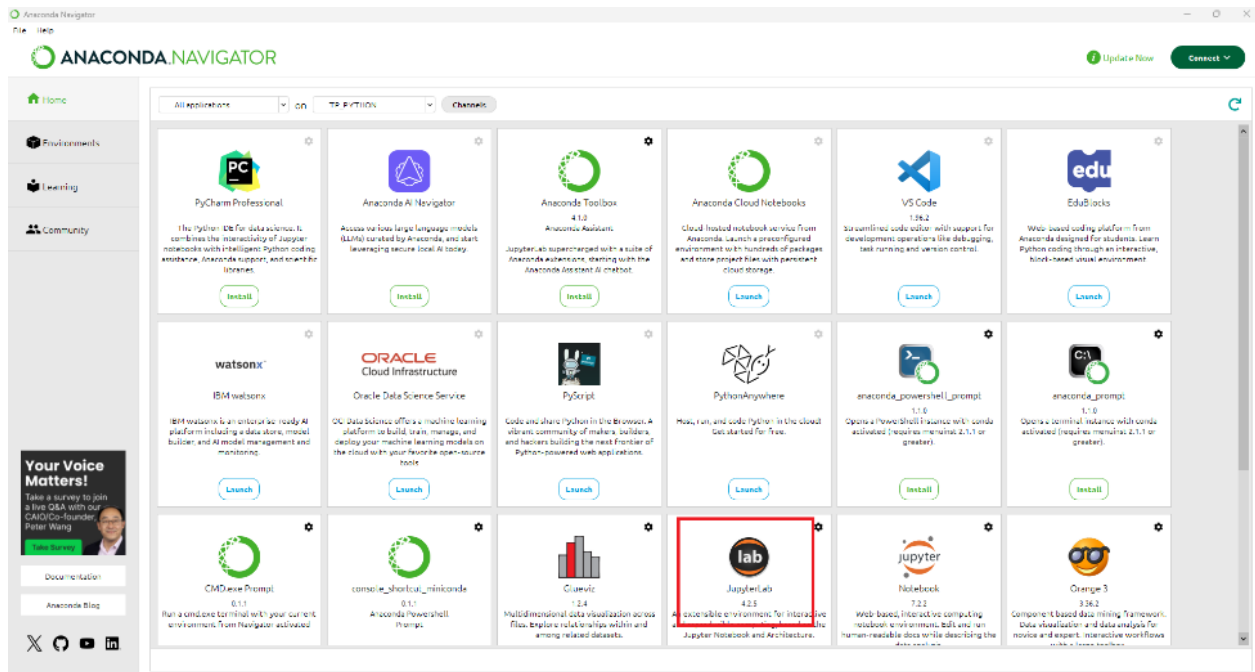
2. Activer l'environnement TP_PYTHON

1. Une fois dans Anaconda Navigator, accédez à l'onglet **Environments** (à gauche).
2. Dans la liste des environnements, sélectionnez **TP_PYTHON** (pré-configuré pour vos TP's).
3. Cliquez sur le bouton **Activate** en haut si ce n'est déjà fait. Sinon, assurez-vous que cet environnement est activé.



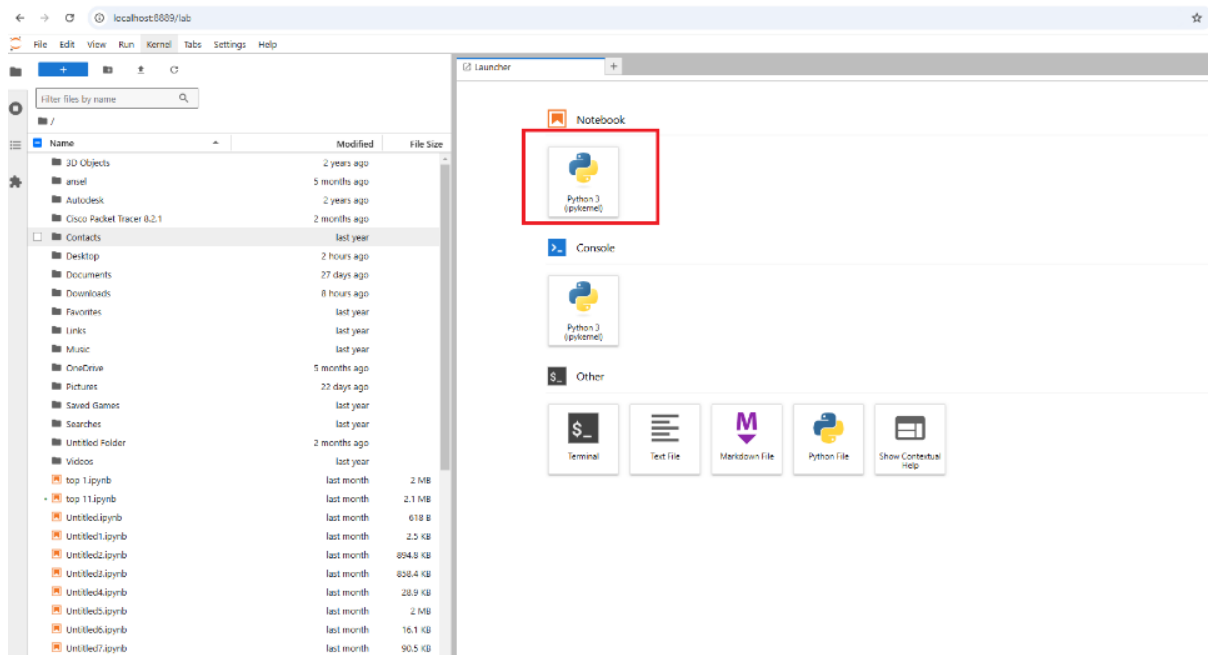
3. Lancer JupyterLab

1. Une fois l'environnement activé, retournez à l'onglet **Home**.
2. Localisez l'application **JupyterLab** dans la liste des outils disponibles.
3. Cliquez sur **Launch** pour ouvrir JupyterLab.



4. Créer un nouveau notebook

1. Dans l'interface de JupyterLab, cliquez sur **File > New > Notebook**.
2. Sélectionnez **Python 3 (ipykernel)** comme noyau pour le notebook.
3. Une fois le notebook créé, enregistrez-le immédiatement avec un nom significatif, par exemple **TP1_Python_Nom_Prénom.ipynb** :
 - Cliquez sur **File > Save As**.
 - Entrez le nom et enregistrez-le dans un emplacement facile à retrouver.



5. Instructions pour réaliser le TP

1. L'ensemble des exercices d'un TP doit être réalisé dans ce **seul notebook**.
2. Organisez le notebook avec des **cellules Markdown** et **cellules de code** pour une présentation claire :
 - Utilisez des titres pour chaque exercice (par exemple : # Exercice 1 : Lecture du fichier, etc.).
 - Ajoutez des explications ou commentaires pour décrire chaque étape du code.
3. Assurez-vous d'exécuter chaque cellule après l'avoir écrite pour valider le bon fonctionnement de votre code.

6. Sauvegarder et télécharger votre travail

1. Enregistrez votre travail régulièrement en cliquant sur l'icône de disquette ou via **File > Save Notebook**.
2. Avant de terminer, vérifiez que toutes les cellules ont été exécutées et que les résultats sont visibles.
3. Téléchargez le fichier final en allant dans **File > Download as > Notebook (.ipynb)** et sauvegardez-le dans votre espace de travail ou envoyez-le vers votre boîte email.

TP1 : La récursivité, le tri et la recherche dichotomique en Python

L'objectif de ce TP est d'une part d'approfondir la prise en main des notions de base du langage python à savoir les types, les séquences de données, les structures conditionnelles et les structures répétitives et d'autre part d'appliquer ses compétences pour concevoir des scripts python des algorithmes de mathématiques basés sur les notions de récursivité, de dichotomie et de tri.

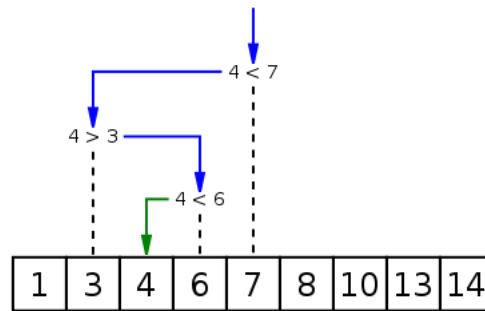
Exercice 1 : La récursivité

- Pour chacun des cas suivant écrire une fonction python récursive :
 1. La fonction **Fact(N)** calculant la factorielle d'un entier naturel **N** (le produit des nombres entiers strictement positifs inférieurs ou égaux à **N**) et retournant un entier positif.
 2. La fonction **Fibo(M)** calculant les **M** premiers éléments de la suite de Fibonacci (c'est une suite d'entiers dans laquelle chaque terme est la somme des deux termes qui le précèdent et les deux premiers termes sont 1 et 1) et retournant l'élément **M** de la suite.
 3. La fonction **PGCD (A, B)** calculant le plus grand diviseur commun de deux nombres entiers positifs **A** et **B** ($A > B > 0$) avec l'algorithme d'Euclide ($A = B.q + r$ sachant que '**q**' est le quotient et '**r**' est le reste de la division de **A** par **B**) et retournant un entier positif. Le principe de fonctionnement de cet algorithme est d'arrêter si **B** = 0 et de rendre la valeur **A**. Sinon, recalculer le reste **r** de la division euclidienne de **a** par **b**. Puis, recommencer avec la mise à jour de **A** et **B** ($A = B$ et $B = r$).
- **Tester** plusieurs fois (au moins 5) chacune de vos fonctions et valider son fonctionnement avec votre enseignant.

Exercice 2 : La recherche dichotomique

La recherche dichotomique sert à chercher un élément **E** dans un tableau trié. Elle commence par comparer cet élément avec la valeur milieu du tableau. Si les valeurs sont égales, la tâche est accomplie, sinon elle recommence dans la moitié du tableau pertinente et cela jusqu'à comparer avec la dernière case si l'élément n'est pas trouvé.

- Ecrire une fonction python **Dichotom(E, L)** qui sert à chercher une valeur **E** donnée dans un tableau **L**. Elle renvoie la valeur si elle est trouvée ou **False** sinon.
- **Tester** plusieurs fois (au moins 5) votre fonction pour chercher un nombre aléatoire **N** saisi par l'utilisateur dans la liste $[0, 1, \dots, 50]$ et valider son fonctionnement avec votre enseignant.
- Un exemple du principe de fonctionnement de la dichotomie est donné sur la figure ci-dessous (recherche de la valeur 4).



Exercice 3 : Les algorithmes de tri

Parmi les algorithmes de tri en mathématiques et en informatique on cite les trois algorithmes ci-dessous.

- **Le tri à bulle** est un algorithme qui tri une liste non ordonnée de valeurs en comparant deux à deux les éléments e_1 et e_2 consécutifs de la liste et en effectuant une permutation si $e_1 > e_2$. Et il continue de trier jusqu'à ce qu'il n'y ait plus de permutation. Pour implémenter cet algorithme, écrire une fonction python **Tri_Bulle(L, L_tri)** qui prend une liste non ordonnée L et renvoie une liste ordonnée dans le sens croissant L_tri.
- **Le tri par sélection** est un algorithme qui tri une liste non ordonnée de valeurs en cherchant le plus petit élément de la liste pour le mettre en premier indice, puis de repartir du second élément et d'aller chercher le nouveau plus petit élément pour le mettre en second indice, etc. Et cela jusqu'au dernier. Pour implémenter cet algorithme, écrire une fonction python **Tri_Select(L, L_tri)** qui prend une liste non ordonnée L et renvoie une liste ordonnée dans le sens croissant L_tri.
- **Le tri par fusion** est un algorithme qui tri une liste non ordonnée de valeurs en divisant en deux moitiés la liste et trier chacune d'entre elles. Ensuite, fusionner les deux moitiés obtenues pour reconstituer la liste finale triée. Pour implémenter cet algorithme, écrire une fonction python **Tri_Fusion (L, L_tri)** qui prend une liste non ordonnée L et renvoie une liste ordonnée dans le sens croissant L_tri.
- **Tester** plusieurs fois (au moins 5) chacune de vos fonctions sur une liste de 20 nombres aléatoires tirés entre 1 et 40 et valider son fonctionnement avec votre enseignant.



TP2 : Le chiffrement de César

Le chiffrement de **César**, également connu sous les noms de chiffrement par décalage ou code de César, est l'une des techniques de cryptographie les plus simples et les plus anciennes. Son nom provient de Jules César lui-même, qui l'aurait utilisé pour transmettre des messages confidentiels à ses généraux. Le principe de fonctionnement de cet algorithme consiste à remplacer chaque lettre d'un message par une autre lettre située à un certain nombre de positions plus loin dans l'alphabet. Ce nombre est appelé **clé de chiffrement** ou **paramètre de décalage**. Notez que le chiffrement conduit à se déplacer de manière circulaire dans l'alphabet, qu'une lettre majuscule (respectivement minuscule) reste chiffrée en majuscule (respectivement en minuscule) et que les caractères non alphabétiques restent inchangés. Par exemple, le chiffrement du message "**Bonjour !**" avec une clé de décalage qui vaut 4 donne le message "**Fsrnsyv !**". Le déchiffrement se fait par la même clé en décalant inversement la position de chaque lettre dans l'alphabet.



L'objectif de ce TP est d'implémenter le chiffrement de César utilisant le langage Python. Durant la séance de TP vous serez amené à concevoir des fonctions élémentaires permettant de réaliser les différents étapes traitant le message clair depuis son acquisition jusqu'à la génération de son chiffrement correspondant dans un sens (partie I) et depuis l'acquisition d'un message chiffré jusqu'à la restitution du message clair correspondant (partie II).

La troisième partie du TP consiste à créer une interface graphique utilisant la librairie Tkinter permettant d'ouvrir un fichier texte, de chiffrer son contenu utilisant une clé saisie par l'utilisateur et de le sauvegarder dans un autre fichier si l'option **Chiffrer le fichier** est choisie. Sion, elle permet d'ouvrir le fichier texte, de déchiffrer son contenu utilisant toujours une clé saisie par l'utilisateur (ou décodée) et de le sauvegarder dans un nouveau fichier si l'option **Déchiffrer le fichier** est choisie.

Pour la bonne structuration de votre travail durant la séance de TP, commencer par créer deux fichiers python (*.py). Le premier sera nommé '**Appli_Chiffrement.py**' et sera utilisé pour sauvegarder le code python de toutes les fonctions que vous allez concevoir. Le deuxième sera nommé '**Test_Appli_Chiffrement.py**' et sera utilisé pour faire tous les tests qui vous seront demandés.

Partie I :

La conception de cet algorithme consiste à le décomposer en actions paramétrés (fonctions et procédures) simples. Il va falloir donc de les mettre en place progressivement comme demandé ci-dessous.

- Définir une fonction de nom **Acquerir_Texte(...)** réalisant l'acquisition d'un texte terminé par un caractère '.' et composé d'au plus 80 caractères (limite non nécessairement atteinte !).
- Définir une fonction de nom **Afficher_Texte(...)** réalisant l'affichage à l'écran d'une suite de caractères enregistrés par ailleurs et fournie en entrée.
- Définir une fonction de nom **Acquerir_Cle(...)** réalisant la lecture de la clé de chiffrement fournie par l'utilisateur et en acceptant comme clé valide que des valeurs entières comprises entre 0 et 25 (bornes incluses).
- Définir une fonction de nom **Chiffre_Texte(...)** réalisant à partir d'une clé donnée et d'un texte clair fournit, le texte chiffré correspondant. Notez que vous pouvez procéder par le principe de traiter le texte caractère par caractère suivant le principe du code de César évoqué dans le cahier des charges.
- Ecrire un programme de test principal pour valider avec votre enseignant le bon fonctionnement de ces diverses fonctions.

Partie II :

On s'intéresse maintenant à l'opération inverse : reconstruire le texte clair à partir d'un texte chiffré. Cette opération est élémentaire si la clé est donné directement ou connue par le récepteur du message. Mais se pose le problème de la détermination de cette clé dans le cas contraire. Pour remédier à ce problème il faut soit connaitre tout le message originale correspondant au chiffré ou une partie (par exemple la première lettre), soit avoir une autre information comme la langue dans laquelle le message a été écrit, etc.

- Définir une fonction de nom **Dechiffre_Texte(...)** en mesure de retrouver le texte clair à partir du texte chiffré et de la valeur de la clé utilisée. Notez que vous pouvez faire appel à la fonction **Chiffre_Texte()** déjà défini en choisissant soigneusement les paramètres d'appel.
- Définir une fonction de nom **Decoder_Cle_V1(...)** en mesure de retrouver la valeur de la clé de chiffrement à partir du texte chiffré et de la valeur du premier caractère alphabétique du texte clair original.
- Si l'utilisateur n'a aucune information sur le message clair, définir une fonction de nom **Decoder_Cle_V2(...)** en mesure de retrouver la valeur de la clé de chiffrement à partir du texte chiffré seulement. Pour cette partie mettez l'hypothèse que le texte a été écrit en Français et que la lettre 'e' est la lettre la plus fréquente dans un texte écrit dans cette langue.
- Compléter le programme de test principal pour valider avec votre enseignant le bon fonctionnement des actions de cette deuxième partie.

Partie III :

Pour mettre en valeur le travail des deux parties précédentes, vous pouvez concevoir une application qui sert à automatiser le chiffrement ou déchiffrement d'un texte saisi par l'utilisateur ou lu à partir d'un fichier et afficher le résultat sur l'écran ou le sauvegarder dans un autre fichier. Pour cela suivre les consignes ci-dessous et revenir au cours si besoin pour créer votre interface graphique utilisant le module Python **Tkinter**.

- Ajouter un widget permettant de choisir le mode de traitement : chiffrement ou déchiffrement.
- Ajouter un widget permettant de choisir la source du texte à traiter : saisie par l'utilisateur ou lu à partir d'un fichier texte.
- Ajouter un widget permettant de choisir le type d'affichage du résultat : affichage sur l'écran ou une sauvegarde dans un fichier texte.
- Ajouter un widget permettant de choisir le type de la clé : saisie par l'utilisateur ou calculée à partir de l'une des fonctions décoder clé V1 ou V2.
- Ajouter un widget permettant de valider le choix de l'utilisateur et d'exécuter l'opération configurée.
- Ajouter d'autres options qui vous sembleront utiles et rendent le fonctionnement de votre application plus performant.
-

The screenshot shows a graphical user interface for a Caesar cipher application. The window is titled "Appli_Chiffrement_César". It contains several interactive elements: a "Type de clé" section with three radio button options (selected, Decoder_Cle_V1, Decoder_Cle_V2) and their respective input fields; a "Mode de traitement" section with two radio button options (selected, Déchiffrement) and a red "Valider" button; a "Source du texte" section with two radio button options (Saisie utilisateur, Fichier texte selected) and their input fields; an "Affichage du résultat" section with two radio button options (Ecran, Fichier texte selected) and its input field; and a large empty rectangular area labeled "Ecran d'affichage".

Exemple de l'application graphique chiffrement de César.

TP3 : La programmation orientée objets en python

Ce TP permet d'appréhender les concepts clés de la programmation orientée objets (POO) utilisant le langage Python tels que la conception de classes avec ses méthodes et ses attributs, l'encapsulation, le polymorphisme, l'héritage et l'héritage multiple en modélisant un schéma UML donné et en respectant un cahier des charges établi. Ensuite, tester ces différents aspects utilisant une classe de test qui simulera le fonctionnement de chaque classe et assurera la gestion des différentes relations connues dans la POO.

Afin d'atteindre ces objectifs, vous allez concevoir et tester progressivement les différentes classes du modèles UML donné sur la figure à la fin de ce TP (*Modèle UML de l'ensemble des classes du projet POO*). Vous commencerez par une classe principale **Personne**. Ensuite, vous ajouterez différentes classes représentant les étapes principales dans la vie d'un être humain de sa jeunesse jusqu'à sa retraite en s'intéressant surtout au côté économique de chaque époque.

Pour la bonne structuration de votre travail durant la séance de TP, commencer par créer deux fichiers python (*.py). Le premier sera nommé **classe_perso.py** et sera utilisé pour sauvegarder le code python de toutes les classes que vous allez concevoir. Le deuxième sera nommé **test_classe_perso.py** et sera utilisé pour faire tous les tests qui vous seront demandés.

Exercice 1 : Organiser et encapsuler son code en classes et méthodes

- Dans le fichier **classe_perso.py** commencer par définir une classe nommée '**Personne**' qui représentera un personnage avec un **nom** et une **position** (x, y) dans un espace 2D.
- Paramétrer le constructeur de cette classe en sorte qu'à la création d'un objet il instanciera le nom et la position d'un personnage.
- Créer une méthode spéciale **__repr__()** qui affichera l'état d'un objet Personne **P**, comme suit : ***Jule est à la position [1, 3]***.
- Dans le fichier **test_classe_perso.py**, créer deux objets **P1** et **P2** de la classe Personne et vérifier leurs instanciation et affichage.
- Créer une méthode **avancer_d(pas)** qui simulera le déplacement du personnage en ligne droite devant lui (de la gauche vers la droite) sur l'axe des abscisses avec un pas égal à '**pas**'.
- De la même manière, créer une méthode **avancer_g(pas)** qui simulera le déplacement du personnage en ligne droite de la droite vers la gauche sur l'axe des abscisses, une méthode **avancer_h(pas)** qui simulera le déplacement du personnage en ligne droite du bas vers le haut sur l'axe des ordonnées et une méthode **avancer_b(pas)** qui simulera le déplacement du personnage en ligne droite du haut vers le bas sur l'axe des ordonnées.
- Taster le fonctionnement de ces méthodes dans votre fichier de test.

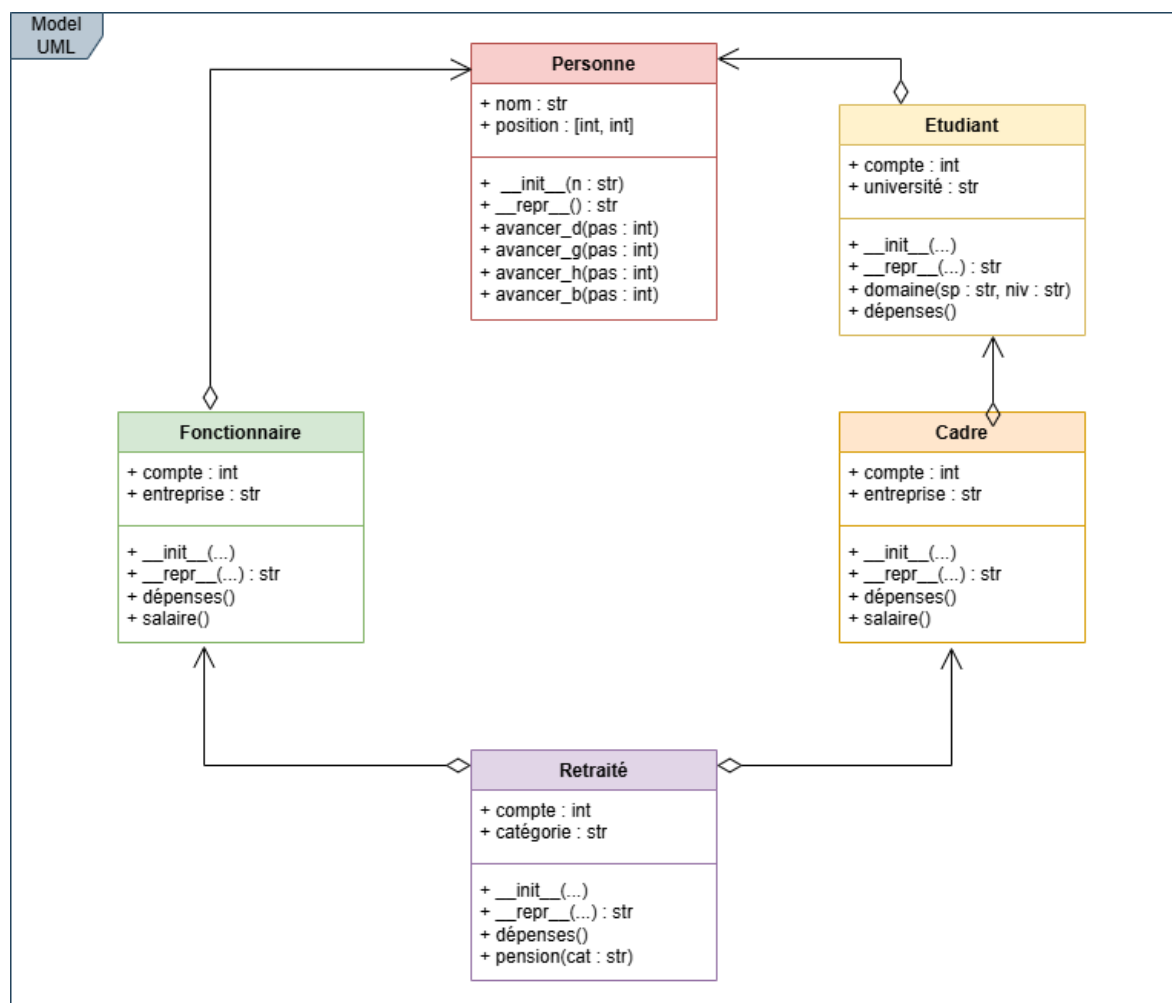
Exercice 2 : Instancier et hériter d'une classe

- Pour avancer dans la conception de votre modèle UML, ajouter dans votre fichier **classe_perso.py** une classe fille **Etudiant** qui héritera des attributs et des méthodes de la classe Personne.
- Compléter le constructeur de cette classe pour déclarer l'héritage et ajouter l'attribut **université** qui indiquera le lieu des études et l'attribut **compte** qui indiquera le solde de son compte bancaire, qui est par défaut de 100 euros.

- Créer une méthode spéciale **__repr__()** qui affichera l'état d'un objet étudiant **E1**, comme suit : ***Jule est étudiant à l'université de Dijon et a sur son compte 100 euros.***
- Taster le fonctionnement de cette classe dans votre fichier de test.
- Ajouter une méthode **domaine(sp, niv)** qui indiquera la spécialité et le niveau des études suivis par t'étudiant.
- Ajouter une méthode **dépenses()** qui débite le compte bancaire d'un étudiant de 50 euros à chaque achat.
- Taster le fonctionnement de ces méthodes dans votre fichier de test.
- Ajouter une troisième classe nommée **Cadre** au fichier **classe_perso.py** et qui héritera des attributs et des méthodes de la classe Etudiant.
- Compléter son constructeur pour déclarer l'héritage et ajouter l'attribut **entreprise** qui indiquera le lieu du travail et l'attribut **compte** qui indiquera le solde de son compte bancaire, qui est par défaut de 2000 euros.
- Créer une méthode spéciale **__repr__()** qui affichera l'état d'un objet cadre **C1**, comme suit : ***César est cadre à la poste et a sur son compte 2000 euros.***
- Taster le fonctionnement de cette classe dans votre fichier de test.
- Ajouter une méthode **dépenses()** qui débite le compte bancaire d'un cadre de 500 euros à chaque achat.
- Ajouter une méthode **salaire()** qui vire un salaire mensuel de 3000 euros sur le compte bancaire d'un cadre.
- Taster le fonctionnement de ces méthodes dans votre fichier de test.
- Ajouter une quatrième classe nommée **Fonctionnaire** au fichier **classe_perso.py** et qui héritera des attributs et des méthodes de la classe Personne.
- Compléter son constructeur pour déclarer l'héritage et ajouter l'attribut **entreprise** qui indiquera le lieu du travail et l'attribut **compte** qui indiquera le solde de son compte bancaire, qui est par défaut de 1000 euros.
- Créer une méthode spéciale **__repr__()** qui affichera l'état d'un objet fonctionnaire **F1**, comme suit : ***Rayane est fonctionnaire à la poste et a sur son compte 1000.***
- Taster le fonctionnement de cette classe dans votre fichier de test.
- Ajouter une méthode **dépenses()** qui débite le compte bancaire d'un fonctionnaire de 200 euros à chaque achat.
- Ajouter une méthode **salaire()** qui vire un salaire mensuel de 1800 euros sur le compte bancaire d'un fonctionnaire.
- Taster le fonctionnement de ces méthodes dans votre fichier de test.

Exercice 3 : Héritage multiple

- Pour définir la relation d'un retraité avec son ancien travail, ajouter une cinquième classe nommée **Retraité** au fichier **classe_perso.py** et qui héritera à la fois des attributs et des méthodes des classes Fonctionnaire et Cadre.
- Compléter son constructeur pour déclarer l'héritage et ajouter l'attribut **catégorie** qui indiquera l'ancien grade du retraité (cadre ou fonctionnaire simple) et l'attribut **compte** qui indiquera le solde de son compte bancaire, qui est par défaut de 5000 euros.
- Créer une méthode spéciale **__repr__()** qui affichera l'état d'un objet retraité **R1**, comme suit : *Alice est retraité, elle était fonctionnaire simple et a sur son compte 5000 euros*.
- Taster le fonctionnement de cette classe dans votre fichier de test.
- Ajouter une méthode **dépenses()** qui débite le compte bancaire d'un retraité de 150 euros à chaque achat.
- Ajouter une méthode **pension(cat)** qui vire un salaire mensuel de 80% de l'ancien salaire d'un retraité sur son compte bancaire selon sa catégorie **cat**.
- Définir deux retraités **R1** ancien cadre et **R2** ancien fonctionnaire et donner le solde de leurs compte bancaire après 3 ans à la retraite sachant qu'ils ont travaillé 30 ans et ils épargnaient 10% de leurs salaires chaque mois durant leurs carrières.



Modèle UML de l'ensemble des classes du projet POO.

TP4 : Traitement d'une image numérique

Cette séance sera consacrée au développement d'une application qui sert à faire du traitement d'une image numérique. L'objectif du TP est de charger et afficher une image à partir d'un fichier. Ensuite, l'utiliser pour extraire des informations, faire des traitements et sauvegarder les résultats des images traitées dans de nouveaux fichiers.

Sachant qu'une image est composée de pixels (points sur l'écran) et chaque pixel a une couleur. Selon le type de codage de couleurs utilisé on peut identifier différentes résolutions. Le cas le plus simple connu est d'utiliser une palette pour chacune des trois couleurs de base (R : rouge, V : vert et B : bleu) et de coder chaque palette sur 1 ou 2 octets. Pour faciliter le travail, vous allez utiliser durant ce TP des images en niveaux de gris (une palette de couleurs) codées seulement sur 8 bits (la valeur de chaque pixel varie entre 0 : noir et 255 : blanc) et de taille 512*512.

Travail à réaliser :

- Créer une fonction nommée **Lire_Image(...)** qui servira à lire l'image à partir d'un fichier '*.png' et sauvegarder son contenu dans un tableau de pixels à une seule dimension et avec la bonne taille (512*512).
- Créer une fonction nommée **Calcul_Histo(...)** qui calculera l'histogramme (le nombre de pixels de chacune des 256 couleurs) des niveaux de gris utilisés dans l'image et retournera un tableau de 256 entiers.
- Créer une fonction nommée **Affiche_Histo(...)** qui affichera l'histogramme calculé par la fonction Calcul_Histo() dans une figure paramétrée avec un titre et un nom pour chacun des axes des abscisse et des ordonnées.
- Créer une fonction nommée **Get_Pixel(...)** qui retournera la valeur d'un pixel en spécifiant en paramètre sa position (l : ligne et c : colonne) dans l'image.
- De même, créer une fonction nommée **Set_Pixel(...)** qui mettra à jour la valeur d'un pixel en spécifiant en paramètre sa position (l : ligne et c : colonne) dans l'image et la nouvelle valeur à prendre par le pixel.
- Créer une fonction nommée **Binarise_Image(...)** qui prendra le tableau d'une image donnée en paramètre et retourne un tableau de même taille en mettant tous les pixels inférieurs à un seuil donné en noir ou en blanc sinon.
- Créer une fonction nommée **Sauvegarde_Image(...)** qui prendra le tableau d'une image donnée en paramètre et l'enregistre dans un fichier image de type '*.png' avec un nom choisi.
- Créer une fonction nommée **Zoom_Zone(...)** qui affichera une partie de l'image en plus gros en spécifiant en paramètre les coordonnées (l, c) du centre de la zone (on se limite à 5 pixels dans chaque direction de la position du pixel choisi, donc un carré de 11*11 pixels) dans l'image d'origine et le facteur de zoom f. Notez par exemple qu'un facteur de zoom f=2 va créer 2*2 pixels de la même couleur que le pixel principal. Pensez à ajouter à l'intérieur de cette fonction une option qui permettra d'afficher la zone de l'image zoomée.
- Créer une fonction nommée **Filtre_Image(...)** qui appliquera un filtre (matrice carrée de taille impaire de valeurs réelles) choisi en paramètre sur le tableau d'une image donnée et retourne un tableau de l'image filtrée de même taille que l'original. Notez que le

filtrage est un traitement classique des images et sert par exemple à détecter les contours des formes de l'image (filtre de Sobel) ou lisser l'image (filtre de Gauss). Le principe de filtrage consiste superposer le filtre choisi sur une partie de l'image et de le centrer sur le pixel à calculer. Ensuite, multiplier élément par élément les valeurs du filtre par les pixels de l'image chevauchée par le filtre et de faire la somme pour trouver la nouvelle valeur du pixel filtré. Enfin, mettre le résultat à la même position dans l'image de sortie. Pour passer au pixel suivant, il faut glisser le filtre d'une position sur la colonne jusqu'à la dernière colonne et recommencer à la première colonne de la ligne suivante et cela jusqu'au dernier pixel de l'image. Par exemple, pour calculer le pixel à la position (3, 9) d'une image filtrée par Gauss, on applique le calcul ci-dessous :

- **Gauss** = $\begin{vmatrix} 0 & 0.1 & 0 \\ 0.1 & 0.6 & 0.1 \\ 0 & 0.1 & 0 \end{vmatrix}$;

- **Sobel** = $\begin{vmatrix} 0 & -0.5 & 0 \\ -0.5 & 0 & 0.5 \\ 0 & 0.5 & 0 \end{vmatrix}$;

$$\begin{aligned} \text{Pixel_filtre}(3, 9) &= 148*0 + 67*0.1 + 61*0 + \\ &111*0.1 + 10*0.6 + 42*0.1 + \\ &71*0 + 78*0.1 + 2*0 \\ &= 35.8 \end{aligned}$$

Vous remarquez que les valeurs résultantes sont de type réel. Pensez donc à les mettre sous forme d'entier non signés entre 0 et 255.

0	...	6	9	10	...
1	95	120	69	0	15
2	218	208	45	10	17
3	200	170	67	61	19
4	153	149	36	42	32
...	3	14	78	2	10
	1	4	69	65	88

Pixel(3, 9) dans l'image filtrée.

0	...	6	7	9	9	10	11	...
1	95	120	52	31	69	45	10	15
2	218	208	210	125	0	0	9	17
3	200	170	198	148	67	61	23	19
4	153	149	127	111	10	42	45	32
...	3	14	10	71	78	2	85	10
	1	4	11	13	69	65	60	88

Calcul du Pixel(3, 9) à partir de l'image originale et du filtre de Gauss.

TP5 : Analyse des données à partir d'un fichier tableur

L'objectif de ce TP est de se familiariser avec la bibliothèque Pandas, un outil puissant pour l'analyse de données en Python. Vous apprendrez à manipuler des tableaux de données (DataFrames), qui permettent de représenter des informations sous forme tabulaire avec des lignes (individus) et des colonnes (variables). Ce TP vous guidera dans les opérations fondamentales telles que la lecture de fichiers tabulés, la manipulation des colonnes et des lignes, la fusion de données, ainsi que l'analyse statistique. Vous découvrirez également comment préparer les données pour des visualisations graphiques en exploitant la compatibilité avec des bibliothèques telles que Matplotlib. Ce TP mettra l'accent sur la gestion efficace des structures de données, essentielles en science des données.

Travail à réaliser :

- Créer une fonction nommée **Lire_Donnees(...)** qui permettra de lire des données à partir d'un fichier **'*.xlsx,'** et de les charger dans une structure de type DataFrame qu'elle retournera.
- Utiliser la fonction précédente pour lire le fichier **'individus.csv'** (à télécharger sur plubel) dans une DataFrame **df1**.
- Afficher les 4 premières lignes, les 3 dernières lignes ainsi que des informations générales sur les colonnes et leurs types du DataFrame df1.
- Remplacer les caractères spéciaux **' ? '** par des valeurs indéfinies **'Nan'** dans df1 puis repérer, compter et afficher les données manquantes (**'Nan'**).
- Créer une fonction nommée **Nettoyer_Donnees(...)** qui servira à analyser un DataFrame reçu en paramètre et supprimer les lignes et les colonnes vides et remplacer les valeurs inconnues (**'Nan'**, cellule vide...) par une valeur par défaut en lien avec les autres valeurs de la même colonne (par exemple par la valeur moyenne pour les colonnes avec des valeurs numériques) et retournera le DataFrame résultant.
- Appliquer la fonction Nettoyer_Donnees() sur le DataFrame df1 et sauvegarder le résultat dans le DataFrame **df2** et l'afficher pour vérifier le résultat.
- Ajouter les labels **'L1, L2, ..., Ln'** aux indexes des lignes et l'entité **'Nom, Prénom, Sexe, Taille (cm), Poids (Kg), Age (ans)'** aux indexes des colonnes de df2.
- Créer une fonction nommée **Ajouter_Colonne_IMC(...)** qui prendra en paramètre un DataFrame et ajoutera une colonne nommée IMC (Indice de Masse Corporelle) calculée à partir des colonnes Taille (cm) et Poids (kg) s'elles existent et retournera le DataFrame résultant. La formule de calcul de l'IMC est : **$IMC = Poids\ (kg) / (Taille\ (m)^2$**).
- Appliquer la fonction Ajouter_Colonne_IMC() sur le DataFrame df2 et sauvegarder le résultat dans le DataFrame **df3** et l'afficher pour vérifier le résultat.
- Créer une fonction nommée **Filtrer_df(...)** qui prendra en paramètre un Dataframe et un index de l'entité des colonnes (taille, poids, sexe...) et retournera dans un autre Dtaframe contenant uniquement les lignes correspondantes aux individus du paramètre choisi.
- Appliquer cette fonction sur df3 pour chercher uniquement les individus de sexe masculin et sauvegarder le résultat dans le DataFrame **df4** et l'afficher pour vérifier le résultat.
- Créer une fonction nommée **Fusionner_Donnees(...)** qui fusionnera deux DataFrames reçus en paramètre et retournera le DataFrame résultant.
- Créer un Dataframe **df5** utilisant Pandas (voir exemple en cours) de même nombre de lignes que df4 et ajouter une activité sportive de votre choix à chaque individu.
- Utiliser la fonction Fusionner_Donnees() pour grouper les deux DataFrames df3 et df5 dans un seul DataFrame **df6** et l'afficher pour vérifier le résultat.

- Créer une fonction nommée **Statistiques_df(...)** qui prendra en paramètre un DataFrame et un index de l'entité des colonnes pour calculer les statistiques de base correspondantes (par exemple : moyenne, médiane, minimum, maximum) et retournera le résultat dans un Dataframe adéquat.
- Appliquer la fonction Statistiques_df() sur df6 pour faire une analyse statistique sur l'âge des individus et sauvegarder le résultat dans le DataFrame **df7** et l'afficher pour vérifier le résultat.
- Créer une fonction nommée **Normaliser_Donnees()** qui prendra en paramètre un DataFrame pour normaliser toutes ses colonnes numériques (taille, poids, age) et retournera le DataFrame résultant. Le calcul de la norme sera basé sur la formule suivante.

$$x_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- Appliquer la fonction Normaliser_Donnees() sur df6 pour normaliser ses données numériques et sauvegarder le résultat dans le DataFrame **df8** et l'afficher pour vérifier le résultat.
- Créer une fonction nommée **Afficher_Distribution(...)** qui prendra en paramètre un DataFrame et un index de l'entité des colonnes et tracera l'histogramme correspondant si les valeurs de la colonne choisie sont numériques.
- Appliquer cette la fonction sur df6 pour afficher la distribution de l'IMC des individus.
- Créer une fonction nommée **Exporter_Donnees(...)** qui sauvegardera un DataFrame dans un fichier de type '*.CSV' puis l'utiliser pour sauvegarder vos DataFrames analysées (df2, df3, df4...) et vérifier la génération des fichiers correspondants.